Program Studi Teknik Informatika
Institut Teknologi Sumatera

MODUL PRAKTIKUM

Sistem Operasi

IF25-12007 | Genap 2025/2026

Mata Kuliah	Sistem Operasi (IF25-12007)
Semester	Genap 2025/2026
Institusi	Institut Teknologi Sumatera

© 2026 — Institut Teknologi Sumatera

TIM PENGAJAR & ASISTEN

Dosen Pengampu

Martin Clinton Tosima Manullang, S.T., M.T., Ph.D.

Ilham Firman Ashari, S.Kom., M.T

I Wayan Wiprayoga Wisesa, S.Kom., M.Kom.

Eko Dwi Nugroho, S.Kom., M.Cs.

Asisten Praktikum

Ahmat Prayoga Sembiring	123140053
Andryano Shevchenko Limbong	123140205
Aziz Kurniawan	122140097
Dito Rifki Irawan	122140153
Ebentua Philippus Limbong	123140086
Elsa Elisa Yohana Sianturi	122140135
Fathan Andi Kartagama	122140055
Miftahul Khoiriyah	123140064
Muhammad Bintang Al-Fasya	123140098
Muhammad Fadhil Zurani	122140146
Reza Chairul Manam	120140086
Stevanus Cahya Anggara	123140038
Varasina Farmadani	123140107
Zakhi Algifari	122140198

Contents

1 Booting dan Framework Sistem Operasi 5

1.1 Tujuan dan Output Praktikum 5

1.1.1 Tujuan Praktikum 5

1.1.2 Output Praktikum 5

1.2 Dasar Teori 6

1.2.1 Pengenalan OS Framework dan Booting 6

1.2.2 Kernel Bare-Bones dan Spesifikasi Multiboot 7

1.2.3 Simulasi Perangkat Keras melalui QEMU 8

1.3 Alat dan Bahan 10

1.3.1 Perangkat Keras 10

1.3.2 Perangkat Lunak (Bahan Praktikum) 10

1.4 Langkah-Langkah Praktikum 11

1.4.1 Persiapan 1: Instalasi Dependensi Dasar dan Docker 11

1.4.2 Persiapan 2: Menjalankan Mesin Terisolasi 12

1.4.3 Sesi 1: Kompilasi Biner OS (Make) 13

1.4.4 Sesi 2: Eksekusi dan Verifikasi di QEMU 14

2 Implementasi Multi-threading 16

2.1 Tujuan dan Output Praktikum 16

2.1.1 Tujuan Praktikum 16

2.1.2 Output Praktikum 16

2.2 Dasar Teori 17

2.2.1 Pengenalan Thread dan Multi-threading 17

2.2.2 Konteks Eksekusi dan Context Switching 17

2.2.3 Thread Control Block / TCB 17

2.2.4 Algoritma Penjadwalan (Scheduling) 18

2.3 Alat dan Bahan 19
2.3.1 Perangkat Keras 19
2.3.2 Perangkat Lunak (Bahan Praktikum) 19
2.4 Langkah-Langkah Praktikum. 20
2.4.1 Sesi 1: Persiapan Lingkungan Kerja 20
2.4.2 Sesi 2: Implementasi Fungsi `thread_create()` 20
2.4.3 Sesi 3: Implementasi Cooperative *Round-Robin* 24
2.4.4 Sesi 4: Kompilasi dan Pengujian QEMU. 25

Bibliography **27**

CHAPTER 1

Booting dan Framework Sistem Operasi

1.1 Tujuan dan Output Praktikum

1.1.1 Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Memahami konsep proses *booting* beserta fungsi komponen penyusun kerangka dasar sistem operasi (seperti *Multiboot Header*, algoritma C/Assembly, dan *Linker Script*).
2. Melakukan proses perakitan (*build*) melalui kompilasi silang (*cross-compilation*) berbasis kontainer terisolasi serta menyimulasikan peluncuran sistem operasinya secara aman menggunakan emulator QEMU.

1.1.2 Output Praktikum

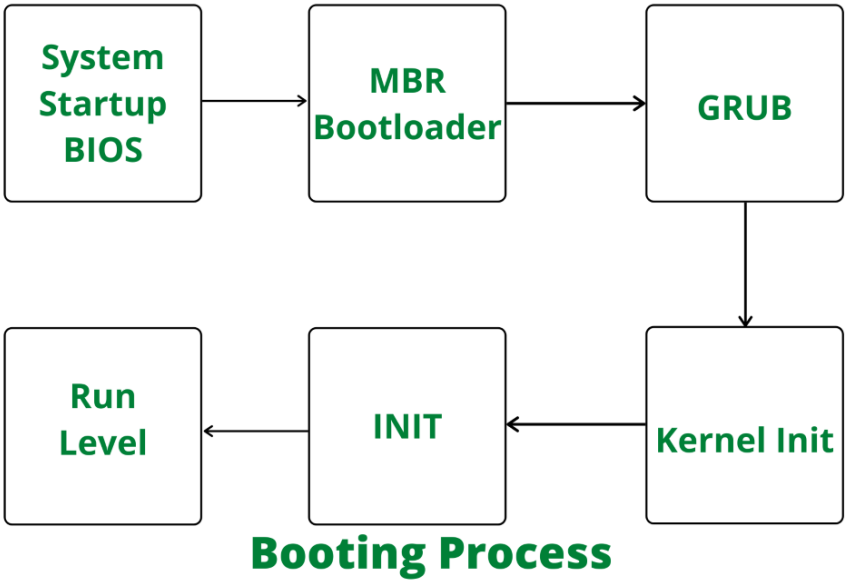
Pada akhir praktikum ini, mahasiswa diharapkan menghasilkan:

- Penjelasan analitis mengenai fungsi dari masing-masing berkas penyusun arsitektur sistem operasi (terutama `boot.asm`, `kernel.c`, dan `linker.ld`).
- Berkas instalasi sistem operasi berformat `.iso` yang berhasil ter-*compile* dan mematuhi spesifikasi standar *bootloader*, beserta tangkapan layar (*screenshot*) bukti sistem sukses melewati siklus *booting* di dalam peramban QEMU.

1.2 Dasar Teori

1.2.1 Pengenalan OS Framework dan Booting

Sistem operasi adalah perangkat lunak utama yang berinteraksi langsung dengan fisik perangkat keras komputer [1]. Saat komputer pertama kali dinyalakan, serangkaian instruksi berurutan yang disebut *booting* akan berjalan. Proses ini diibaratkan seperti lari estafet; satu program berukuran kecil akan memanggil program lain yang ukurannya sedikit lebih besar, berlanjut terus hingga sistem operasi secara utuh berhasil mengambil alih kendali komputer.



Gambar 1.1: Visualisasi proses booting komputer secara berurutan

Berdasarkan ilustrasi diagram pada Gambar 1.1, tahapan *booting* secara umum dapat dijabarkan menjadi 6 langkah sederhana:

1. **System Startup BIOS**: Saat tombol *power* ditekan, komputer akan menjalankan program bawaan mesin bernama BIOS. BIOS bertugas mengecek apakah komponen fisik (seperti RAM dan *keyboard*) sudah berfungsi normal.
2. **MBR Bootloader**: Setelah fisik dipastikan aman, BIOS akan mencari instruksi pertama di ujung memori penyimpanan (disk) yang lokasinya disebut MBR. MBR ibarat pintu masuk menuju program selanjutnya.

3. **GRUB**: MBR kemudian memanggil program pengelola *boot* yang lebih pintar, salah satunya adalah GRUB. GRUB inilah yang tahu persis di mana sistem operasi kita berada, lalu memuatnya ke memori utama (RAM).
4. **Kernel Init**: Setelah dimuat ke RAM, inti dari sistem operasi (disebut *Kernel*) mulai terbangun. Kernel secara resmi mengambil alih kendali komputer sepenuhnya dari tangan GRUB.
5. **INIT**: Kernel kemudian menyalakan satu proses utama penggerak yang disebut `INIT`. Proses `INIT` inilah yang bertugas membangunkan fitur-fitur maupun layanan pendukung lain di dalam OS.
6. **Run Level**: Ini adalah kondisi garis finis. Pada tahap *Run Level*, komputer sudah "sadar" sepenuhnya (misalnya berhasil masuk ke layar Terminal/Desktop) dan siap menerima interaksi dari pengguna.

1.2.2 Kernel Bare-Bones dan Spesifikasi Multiboot

Mengembangkan sistem operasi mensyaratkan kerangka dasar (*bare-bones*) yang dapat beroperasi tanpa dukungan sistem operasi lain [2]. Karena berstatus independen dan tidak menggunakan modul fungsi dari *standard library*, inisialisasi awal pada perangkat keras komputer wajib diserahkan kepada komponen antarmuka yang sangat spesifik.

Standar Multiboot pada Bootloader

Agar perangkat *bootloader* utama (seperti GRUB) bersedia menemukan dan memuat kernel spesifik OS ke dalam memori, kerangka OS didesain wajib mematuhi ketentuan dari **Spesifikasi Multiboot** (GNU). Berdasarkan spesifikasi instruksi ini, berkas kompilasi biner awal harus menyuntikkan deklarasi *Multiboot Header* di bagian awal blok *header* berkas (*file format*).

Tanda *header* perizinan ini ditulis menggunakan bahasa Assembly (sebagaimana dideklarasikan melalui `boot.asm`) dengan menyertakan *magic number* unik berbasis bilangan heksadesimal bernilai tetap (`0x1BADB002`) [3]. Kehadiran nomor ajaib ini menjadi patokan (*signature*) identifikasi GRUB untuk mengenalinya sebagai OS terstandarisasi, sebelum perannya meredup dan mengandalkan proses siklus selanjutnya pada fungsi `main()` program bahasa terapan C (`kernel.c`).

Manajemen Relokasi dengan Linker Script

Pada arsitektur OS *bare-metal*, penetapan perihal letak tatanan alamat memori tidak ditangani *loader* dinamis. Ketiadaan itu mensyaratkan eksistensi berkas *Linker Script* (`linker.ld`) untuk secara tegas mendikte alokasi perpindahan posisi kode memori [4]. Pada sistem berbasis arsitektur x86, kernel OS umumnya diamanatkan agar diload ke ruang memori fisik minimal berawal di unit `1M` (`0x100000`), membiarkan ruang memori di bawah 1MB tetap diakses eksklusif sebagai alokasi sakral konfigurasi antarmuka untuk BIOS (*Basic Input/Output System*) maupun Vesa-Graphic peranti tampilan visual perangkat.

1.2.3 Simulasi Perangkat Keras melalui QEMU

Setelah semua bagian kode dipersatukan menjadi satu format instalasi kepingan siap-boot (`os.iso`), eksekusi *host-machine* pada komputer secara telanjang patut dihindari. Komputasi pengujian OS sangat sarat atas probabilitas kekacauan yang sanggup merusak alur partisi (*storage*) mesin *host* sesungguhnya.

Keseluruhan perancangan diuji mutlak dalam isolasi peranti **QEMU**. Emulator x86 ini membentangkan platform simulasi lengkap mulai dari abstraksi CPU, register internal, kepingan Random Access Memory, hingga konfigurasi Serial-Port. Pemanfaatan QEMU menjadi prasyarat esensial guna mengekstrak pantauan alur *log-sequence* operasi komputasi saat prosedur (*Booting*) terjadi dari menit paling awal secara transparan, serta memudahkan rekam jejak apabila fungsi sinkronisasi (seperti penentuan jadwal *task-scheduler*) menuai kecacatan algoritma teknis (*Crash-Faults*).

Tabel 1.1 merangkum hal-hal penting agar pembentukan konsep di atas lebih mudah ditinjau di modul ini.

Tabel 1.1: Tabel Ringkasan Dasar Teori

Konsep / Tools	**Penjelasan Singkat**
Bootloader (GRUB)	Program di sektor awal mesin yang membaca aturan GNU Multiboot dan membangunkan CPU arsitektur 32-bit.
`boot.asm`	Program instruksi mesin (*Assembly*) bervolume hemat yang merengkuh (*Entry-Point*) izin kendali booting OS.
Linker Script (`linker.ld`)	Merupakan set aturan logis tata letak posisi kode memori spesifik di RAM kernel (contoh blok: `0x100000`).
Multiboot Magic Number	Penanda (*signature*) biner wajib penyuntingan GNU untuk menyatakan bahwa berkas bersangkutan diakui Bootloader.
QEMU	Sebuah perangkat lunak berbasis *emulator* mandiri nan aman membedah pengujian alur OS instrumen format .iso.

1.3 Alat dan Bahan

1.3.1 Perangkat Keras

- PC atau Laptop dengan arsitektur prosesor x86/x64.
- Memori minimal 4GB RAM dan kapasitas penyimpanan sekunder yang memadai.

1.3.2 Perangkat Lunak (Bahan Praktikum)

- Sistem operasi *host* (Windows, Linux, atau macOS).
- Text Editor atau IDE (misalnya Visual Studio Code).
- Docker (Docker Desktop atau Docker Engine) untuk proses kompilasi (*cross-compiling*) di lingkungan terisolasi.
- QEMU Emulator (khususnya paket `qemu-system-i386` atau `qemu-system-x86_64`) sebagai mesin virtual.
- Paket kode dasar praktikum (*Starter Code OS 32-bit*) yang dikloning/diunduh dari repositori GitHub: https://github.com/Aziz097/starter-code-os32.

1.4 Langkah-Langkah Praktikum

1.4.1 Persiapan 1: Instalasi Dependensi Dasar dan Docker

Sebelum mengolah *image*, Anda perlu memasang pustaka pengembangan (*multilib compiler*, QEMU, dsb) serta *engine* Docker di mesin *host* utama Anda (khususnya untuk pengguna lingkungan basis distribusi Ubuntu/Debian).

1. Buka Terminal dan perbarui indeks *repository*, lalu pasang dependensi *cross-compile* x86 dan QEMU:

```
sudo apt update
sudo apt install -y build-essential gcc-multilib nasm \
                    qemu-system-x86 grub-pc-bin xorriso

```

Kode 1.1: Instalasi pustaka dasar dan perangkat virtual QEMU

2. Selanjutnya, pastikan Docker terpasang di sistem. Jika belum, instal Docker Engine:

```
sudo apt install -y docker.io

```

Kode 1.2: Instalasi Docker Engine

3. (Opsional namun disarankan) Tambahkan *user* Anda ke grup `docker` agar kelak terbebas dari awalan `sudo` ketika mengeksekusi komando Docker.

```
sudo usermod -aG docker $USER

```

Kode 1.3: Konfigurasi hak otorisasi akses pengguna Docker

Catatan: Restart atau relog Terminal Anda agar perubahan perizinan grup ini langsung terbaca.

1.4.2 Persiapan 2: Menjalankan Mesin Terisolasi

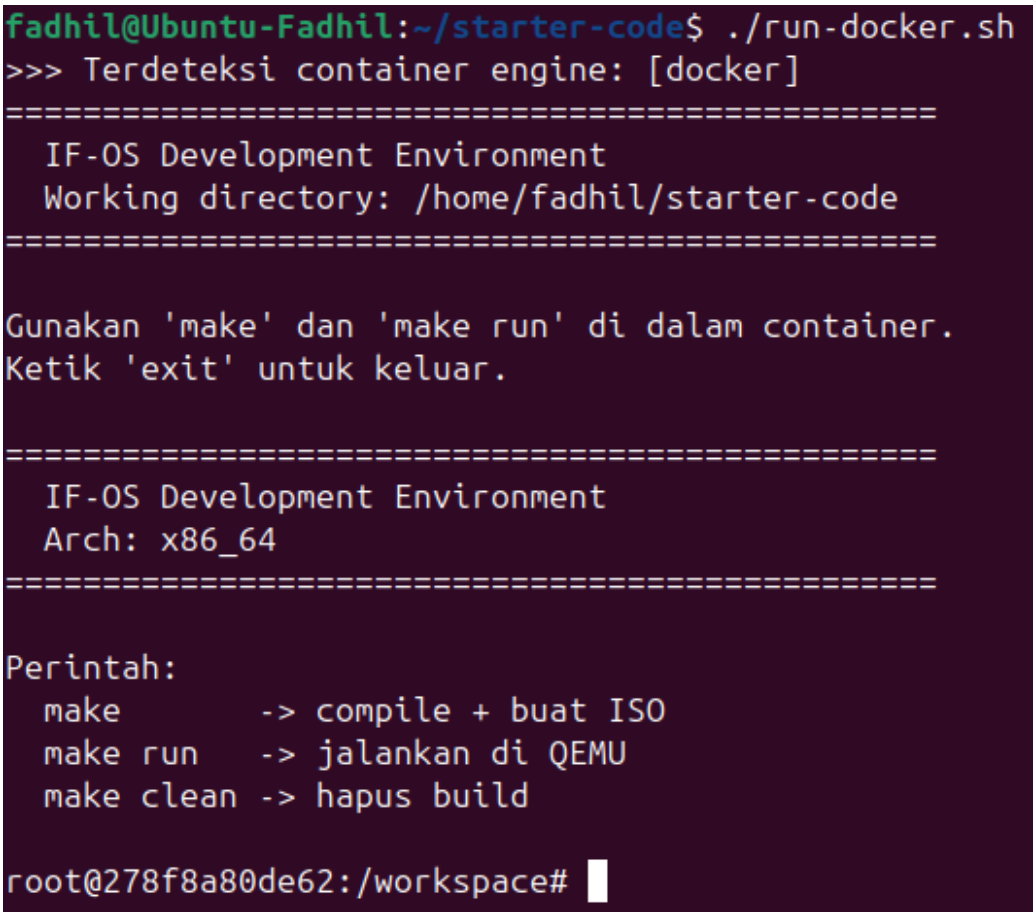
Langkah ini menyiapkan ruang kerja terisolasi agar *tools* kompilasi dan dependensi sistem operasi tidak mengotori mesin *host* (laptop) Anda.

1. Buka terminal, pastikan posisi Anda sudah berada di dalam folder proyek `starter-code-os32-main`, lalu terjun ke mode simulasi lingkungan dengan mengeksekusi *script runner*-nya:

```
# Di sistem Windows (menggunakan Git Bash/WSL) atau Linux/macOS:
./run-docker.sh

```

Kode 1.4: Perintah meluncurkan container Docker



Gambar 1.2: Terminal masuk secara interaktif dengan indikator berpindahnya ruang kendali menjadi status `root@...:/workspace#` di dalam kontainer `ifos-dev`

2. Skrip tersebut akan merakit atau menarik (*pull*) peranti kontainer secara otomatis, ditandai dari tampilan baris *working directory* Anda.

1.4.3 Sesi 1: Kompilasi Biner OS (Make)

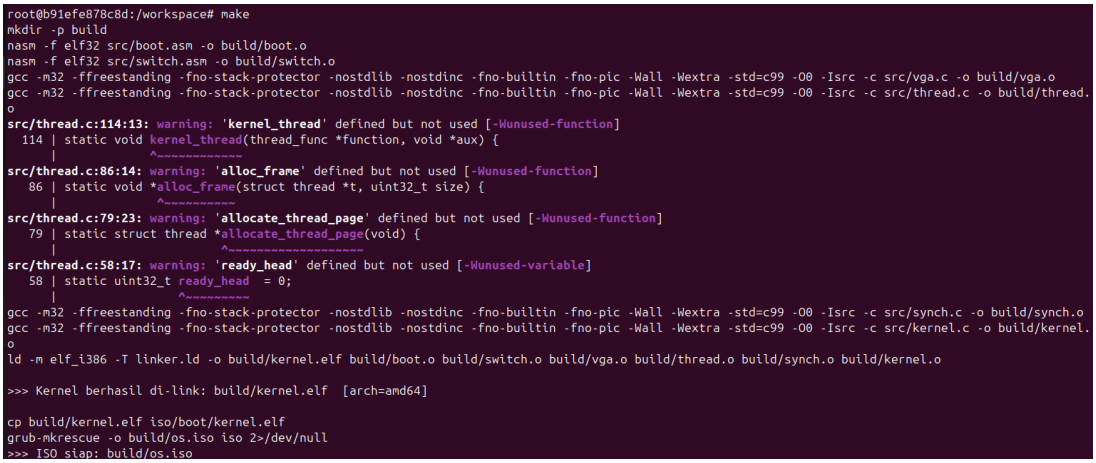
Di dalam lingkungan terisolasi ini, Anda akan menjahit (*linking*) susunan berkas instruksi Assembly (`boot.asm`) dan C (`kernel.c`) menjadi satu *file* instalasi terpadu (`os.iso`).

1. Di dalam *shell* kontainer Docker, periksa isi folder kerja menggunakan perintah `ls`. Secara mendasar, *folder* ini adalah pantulan sikronisasi repositori Anda di mesin *host* Anda.
2. Lakukan kompilasi kode biner sumber dan menautkannya ke instruksi Multiboot GRUB melalui otomasi perintah sistem `make`:

```
make

```

Kode 1.5: Perintah kompilasi berkas instruksi OS



Gambar 1.3: Laporan suksesnya perintah *make* pada *shell* menautkan objek program C hingga *linker* GRUB biner `os.iso`

3. Perhatikan teks keluaran (*output*) kompilator. Jika urutan berhasil, fungsi-fungsi instruksi selesai diproses dan Anda mendapat validasi bertuliskan: `»> ISO siap: build/os.iso`.

1.4.4 Sesi 2: Eksekusi dan Verifikasi di QEMU

Tahapan penutup modul ini adalah menguji coba berkas *bare-metal* OS `.iso` tersebut ke dalam peranti simulator *hardware* berarsitektur x86 bernama emulator QEMU.

1. Keluar dari terminal kontainer Docker (dengan mengetik `exit`) atau buka jendela Terminal baru di mesin *host* asli Anda. Pastikan posisi direktori Anda berada di folder proyek `starter-code-os32-main`.
2. Laksanakan tes pembuka eksekusi skrip emulator dengan menjalankan perintah *make* berikut di mesin *host* Anda (yang sudah diinstal QEMU pada Persiapan 1):

```
make run

```

Kode 1.6: Perintah memacu virtualisasi QEMU meluncurkan file OS biner

3. Jendela layar QEMU (atau konsol emulator) akan muncul dan mengambil alih tampilan dengan antarmuka *Bootloader* fiktif GRUB. *Kernel loader* selanjutnya akan dipandu masuk melalui proses *booting* riil ke antarmuka utama sistem operasi.
4. Oleh karena fitur fungsional komputasi *Thread Scheduling* saat ini masih dirancang kosong secara alur *stub* dalam struktur inti *source code*, wajar apabila OS mencetak log luaran yang bermuara tertahan akibat pesan peringatan berikut (menandakan skema *booting* sudah berhasil dimuat secara mulus):

```
===================================
  IF-OS  |  Praktikum Sistem Operasi
===================================

Inisialisasi sistem thread...
Membuat thread A, B, C...
Memulai scheduling...

ERROR: kernel_main() kembali dari thread_block()!

```

Kode 1.7: Pesan luaran *Thread block* sebagai bukti akhir keberhasilan simulasi di QEMU

CHAPTER 2

Implementasi Multi-threading

2.1 Tujuan dan Output Praktikum

2.1.1 Tujuan Praktikum

Tujuan yang diharapkan dapat dicapai oleh mahasiswa adalah:

1. Memahami arsitektur perangkat lunak dalam manajemen *thread* (termasuk abstraksi *Thread Control Block*) dan algoritma penjadwalan (*scheduling*) pada tingkat inti sistem operasi (*kernel-level*).
2. Mempraktikkan implementasi logika *Context Switching* untuk memfasilitasi eksekusi *multi-threading* melalui penyimpanan dan pemuatan status register CPU (memanfaatkan manipulasi bahasa tingkat rendah/Assembly).

2.1.2 Output Praktikum

Hasil luaran yang diekspektasikan adalah kelengkapan instrumen berikut:

- Implementasi fungsional dari modifikasi berkas sumber (khususnya penyesuaian pada logika `thread.c`, `switch.asm`, dan `scheduler.h`) yang terbukti mampu mengompilasi rekayasa antrean *thread*.
- Tangkapan layar (*screenshot*) keluaran terminal dari emulator QEMU yang memvalidasi bahwa beberapa *thread* (contoh: Thread A, B, dan C) sukses berjalan saling bergantian secara sinkron mengambil alih siklus CPU.

2.2 Dasar Teori

2.2.1 Pengenalan Thread dan Multi-threading

Dalam sistem komputasi, *thread* adalah unit instruksi terkecil yang dapat dijadwalkan oleh sistem operasi [1]. Jika sebuah proses diibaratkan sebagai program yang sedang berjalan dan memiliki memori sendiri, maka *thread* adalah entitas eksekusi operasional di dalam proses tersebut. Lingkungan OS *bare-metal* merintis kerangka *multi-threading* agar dapat mensimulasikan penanganan banyak fungsi bahasa C secara bersamaan (konkurensi) [5]. Secara konseptual, rutinitas pembuatan dan pengelolaan *thread* dikonstruksi guna mendistribusikan beban kerja secara seimbang sehingga mendongkrak efisiensi utilitas prosesor.

2.2.2 Konteks Eksekusi dan Context Switching

Konteks eksekusi merujuk pada seluruh status kumpulan register dari prosesor (seperti EAX, EBX, EBP, dan ESP pada perangkat arsitektur x86) yang dikenakan oleh sebuah *thread* pada momentum waktu tertentu [4]. Ketika sistem operasi hendak memberhentikan satu *thread* lalu beralih ke giliran antrean eksekusi milik *thread* yang lain, OS wajib menukarkan keadaannya lewat taktik perpindahan *Context Switching*.

Karena kerumitan modifikasi letak tumpukan memori (*stack*) amat bergantung pada wujud arsitektur silikon sistem biner, prosedur krusial ini umumnya direpresentasikan murni melalui arahan instruksi pada level *Assembly*. Operasi tersebut mutlak ditugaskan untuk mengekstrak simpanan (*Push*) dari *thread* lama seraya mengangkat muatan register (*Pop*) profil milik antrean *thread* pengganti.

2.2.3 Thread Control Block / TCB

Agar sistem pengontrol sanggup menjaga identitas serta alokasi titik perhentian terakhir masing-masing *thread*, direkayasa sebuah skema pelacak terstruktur dinamakan *Thread Control Block* (TCB) [1]. Menggunakan struktur data mandiri (seperti halnya tipe rekaman mendasar pada bahasa C), blok TCB secara dedikatif

membungkus parameter-parameter memori berikut:

- **Thread ID**: Angka unit unik pengenal subyek yang tak boleh identik.
- **Status Thread**: Peta kondisi berjalannya instruksi pada suatu *thread* (contoh: status sedang fasif siap diterjunkan/*Ready*, status mandek/*Blocked*, atau sedang berjalan/*Running*).
- **Stack Pointer (ESP)**: Penunjuk referensi baris alamat terakhir yang terekam pada tumpukan (*stack*) ketika interupsi perpindahan prosesnya berlangsung.

2.2.4 Algoritma Penjadwalan (Scheduling)

Siklus perancangan eksekusi CPU dikoordinasi secara tersentralisasi oleh modul OS benama *Scheduler* [1]. Esensi utama dari algoritma komponen ini bertugas menyeleksi manakah prioritas *thread* yang menduduki gerbang *Ready* pada antrean sistem, agar mendapat wewenangnya dialihkan ke tingkatan fase *Running*. Konsep paradigma fundamental penyeleksian dijabarkan melalui dua cara:

1. **Cooperative Scheduling**: Sebuah *thread* secara sadar membiarkan algoritma lain ganti mengeksekusi CPU dengan inisiatifnya menyerah (seperti metode inkuiri `yield()` atau tertahan menunggu sumber daya antarmuka selesai).
2. **Preemptive Scheduling**: Sistem operasi menyetel batasan pencacah peranti keras layaknya *Timer Interrupt*, berfungsi untuk merampas status dominasi algoritma tanpa ampun dari instruksi *thread* yang sedang menduduki memori apakala batasan waktunya telah dirasa mencukupi.

Tabel 2.1 memberikan intisari terminologi vital terkait landasan teori *multithreading*.

Tabel 2.1: Tabel Ringkasan Dasar Teori Multi-threading

Konsep	Penjelasan Singkat
Thread	Unit eksekusi fungsional terkecil di dalam OS.
Context Switch	Tindakan rekam-simpan silang arsitektur register satu *thread* dengan *thread* cadangan lainnya.
TCB	Struktur blok tipe data spesifik pengingat status profil *thread*.
Scheduler	Organ algoritma OS yang mendikte hak pembagian waktu kendali CPU atas *thread*.

2.3 Alat dan Bahan

2.3.1 Perangkat Keras

- PC atau Laptop dengan arsitektur prosesor basis *host* x86/x64 yang mendukung kelanjutan kompilasi emulator.
- Memori minimal 4GB RAM ruang eksekusi.

2.3.2 Perangkat Lunak (Bahan Praktikum)

- OS *Host* utama (Windows dengan WSL/GitBash, Linux, atau macOS).
- Text Editor (misalnya Visual Studio Code) untuk menyunting berkas kode bahasa C dan *Assembly*.
- Instalasi Docker Engine (`docker.io`) untuk menyalakan lingkungan isolasi kompilasi.
- Emulator piranti keras x86: `qemu-system-x86` (atau instalasi QEMU setara) yang akan mengeksekusi binari hasil rakitan OS.
- Direktori *Starter Code* OS 32-bit yang sudah disiapkan sebelumnya (berisi direktori `src` dengan kerangka lengkap `kernel.c`, `thread.h`, `thread.c`, `switch.asm`, `scheduler.h`, dsb).

2.4 Langkah-Langkah Praktikum

2.4.1 Sesi 1: Persiapan Lingkungan Kerja

Langkah pertama adalah menyiapkan direktori *Starter Code* dan masuk ke dalam kontainer Docker.

1. Buka terminal dan pastikan Anda berada di dalam folder `starter-code-os32-main`.
2. Masuk ke lingkungan Docker dengan menjalankan perintah:

```
./run-docker.sh

```

Kode 2.1: Masuk ke lingkungan kontainer

3. Buka berkas `src/thread.c` pada teks editor. Berkas ini adalah file utama yang akan dimodifikasi.

2.4.2 Sesi 2: Implementasi Fungsi `thread_create()`

Fungsi `thread_create()` bertugas menyiapkan *Thread Control Block* (TCB) dan *stack* awal untuk *thread* baru. Buka `thread.c`, cari fungsi pengisian `thread_create`, dan ikuti langkah berikut:

1. **Langkah 1 (Alokasi TCB):** Gunakan `allocate_thread_page()` untuk alokasi memori *thread*. Jika berhasil, panggil `init_thread(t, name, priority)` dan berikan ID dengan `t->tid = allocate_tid()`.

```c
/* 1. Alokasi page */
    struct thread *t = allocate_thread_page();
    if (!t) return TID_ERROR;

    /* 2. Init TCB */
    init_thread(t, name, priority);
    t->tid = allocate_tid();
```

Gambar 2.1: Implementasi Alokasi TCB pada fungsi thread_create()

2. **Langkah 2 (Frame 1 — `kernel_thread_frame`):** Buat variabel *frame* ini menggunakan `alloc_frame()`. Isi `eip` dengan `NULL`, lalu isi `function` dan `aux` sesuai dengan argumen parameter yang dikirim.

```c
/* 3. Frame 1: kernel_thread_frame */
struct kernel_thread_frame *kf = alloc_frame(t, sizeof *kf);
kf->eip      = NULL;
kf->function = function;
kf->aux      = aux;
```

Gambar 2.2: Implementasi Frame 1 pada fungsi thread_create()

3. **Langkah 3 (Frame 2 — `switch_entry_frame`):** Buat variabel *frame* ini dengan alokasi di bawah Frame 1. Isi `eip` dengan memanggil alamat fungsi `kernel_thread`.

```c
/* 4. Frame 2: switch_entry_frame */
struct switch_entry_frame *ef = alloc_frame(t, sizeof *ef);
ef->eip = (void (*)(void))kernel_thread;
```

Gambar 2.3: Implementasi Frame 2 pada fungsi thread_create()

4. **Langkah 4 (Frame 3 — `switch_threads_frame`):** Buat variabel *frame* terbawah ini. Isi variabel `eip` dengan `switch_entry`, lalu set nilai `ebp`, `ebx`, `esi`, dan `edi` menjadi `0`.

```c
/* 5. Frame 3: switch_threads_frame */
struct switch_threads_frame *sf = alloc_frame(t, sizeof *sf);
sf->eip       = switch_entry;
sf->ebp = sf->ebx = sf->esi = sf->edi = 0;
sf->cur_dummy = sf->next_dummy = NULL;
```

Gambar 2.4: Implementasi Frame 3 pada fungsi thread_create()

5. **Langkah 5 (Penyelesaian):** Panggil `thread_unblock(t)` agar dimasukkan ke dalam antrean *ready*, kemudian lakukan *return* id *thread* tersebut (`t->tid`).

```
/* 6. Masuk antrian */
thread_unblock(t);
return t->tid;
```

Gambar 2.5: Langkah Penyelesaian pada fungsi thread_create()

6. Pastikan kerangka utuh fungsi Anda menyerupai format referensi ini:

```c
tid_t thread_create(const char *name, int priority,
                    thread_func *function, void *aux) {
    /* 1. Alokasi page */
    struct thread *t = allocate_thread_page();
    if (!t) return TID_ERROR;

    /* 2. Init TCB */
    init_thread(t, name, priority);
    t->tid = allocate_tid();

    /* 3. Frame 1: kernel_thread_frame */
    struct kernel_thread_frame *kf = alloc_frame(t, sizeof *kf);
    kf->eip      = NULL;
    kf->function = function;
    kf->aux      = aux;

    /* 4. Frame 2: switch_entry_frame */
    struct switch_entry_frame *ef = alloc_frame(t, sizeof *ef);
    ef->eip = (void (*)(void))kernel_thread;

    /* 5. Frame 3: switch_threads_frame */
    struct switch_threads_frame *sf = alloc_frame(t, sizeof *sf);
    sf->eip       = switch_entry;
    sf->ebp = sf->ebx = sf->esi = sf->edi = 0;
    sf->cur_dummy = sf->next_dummy = NULL;

    /* 6. Masuk antrian */
    thread_unblock(t);
    return t->tid;
}
```

Gambar 2.6: Fungsi utuh `thread_create()` secara keseluruhan

2.4.3 Sesi 3: Implementasi Cooperative *Round-Robin*

Bagian ini mengatur agar *thread* dapat saling bergantian menggunakan wewenang CPU. Masih di dalam file `thread.c`:

1. **Modifikasi `thread_yield()`:** Ambil *thread* yang saat ini berjalan menggunakan `thread_current()`. Ubah statusnya menjadi `THREAD_READY`. Masukkan *thread* ini ke dalam antrean `ready_queue` pada indeks `ready_tail`. Geser urutan `ready_tail`, tambahkan jumlah `ready_count`, dan akhiri dengan memanggil fungsi `schedule()`.

```c
void thread_yield(void) {
    struct thread *cur = thread_current();
    cur->status = THREAD_READY;
    ready_queue[ready_tail] = cur;
    ready_tail  = (ready_tail + 1) % (MAX_THREADS * 2);
    ready_count++;
    schedule();
}
```

Gambar 2.7: Implementasi fungsi `thread_yield()`

2. **Modifikasi `next_thread_to_run()`:** Fungsi ini berjalan secara metode FIFO (*First-In-First-Out*). Jika antrean kosong (`ready_count` bernilai 0), kembalikan `initial_thread`. Jika isi antrean tersedia, ambil *thread* dari urutan depan (`ready_queue[ready_head]`), geser letak indeks depan `ready_head`, kurangi nilai hitung `ready_count`, lalu *return thread* yang diambil tersebut.

```c
struct thread *next_thread_to_run(void) {
    if (ready_count == 0) return initial_thread;
    struct thread *next = ready_queue[ready_head];
    ready_head  = (ready_head + 1) % (MAX_THREADS * 2);
    ready_count--;
    return next;
}
```

Gambar 2.8: Implementasi fungsi `next_thread_to_run()`

2.4.4 Sesi 4: Kompilasi dan Pengujian QEMU

Setelah seluruh kode di `thread.c` selesai, jalankan pengujian sistem untuk membuktikan bahwa OS sudah sanggup menjalankan perpindahan waktu *thread*.

1. Buka kembali terminal yang berada di dalam lingkungan Docker, lalu jalankan

perintah kompilasi:

```
make

```

Kode 2.2: Perintah build pada terminal Docker

2. Setelah proses kompilasi selesai (*error* tidak muncul), jalankan QEMU emulator secara langsung menggunakan perintah di bawah ini:

```
make run

```

Kode 2.3: Menjalankan OS pada terminal Docker

3. Perhatikan layar simulator QEMU. Anda harus masuk dalam putaran sistem *infinite loop* yang memperlihatkan ketiga *thread* yang dibuat mencetak teks (Thread A, Thread B, dan Thread C) bergantian per waktu putaran:

```
====================================
  IF-OS  |  Praktikum Sistem Operasi
====================================

Inisialisasi sistem thread...
Membuat thread A, B, C...
Memulai scheduling...

[Thread A] tick 0
[Thread B] tick 0
[Thread C] tick 0
[Thread A] tick 1
[Thread B] tick 1
...

```

Kode 2.4: Ekspektasi output sistem penjadwalan Thread di latar QEMU

Bibliography

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Wiley, 2018.

[2] OSDev Community. (2026) Bare bones - osdev wiki. [Online]. Available: https://wiki.osdev.org/Bare_Bones

[3] Free Software Foundation, *Multiboot Specification version 0.6.96*, 2026. [Online]. Available: https://www.gnu.org/software/grub/manual/multiboot/multiboot.html

[4] R. E. Bryant and D. R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed. Pearson, 2015.

[5] B. Pfaff *et al.*, *Pintos Instructional Operating System*, Stanford University, 2004.